

Segmentación y paginación de memoria

La gestión de memoria es una de las funciones fundamentales de los sistemas operativos, ya que permite administrar de manera eficiente el espacio disponible en la memoria principal y asignarlo a los diferentes procesos que se ejecutan en un sistema. Entre las principales técnicas utilizadas para esta gestión se encuentran la segmentación y la paginación.

La segmentación permite organizar un programa en diferentes áreas lógicas, como código, datos y pila, sin necesidad de que estas áreas se encuentren ubicadas de forma consecutiva en la memoria física. Por otro lado, la paginación divide la memoria en unidades de tamaño fijo, permitiendo que las diferentes partes de un proceso puedan ubicarse en distintos espacios de la memoria.

En la clase analizada se estudia principalmente el funcionamiento de la segmentación, su relación con las tablas de descriptores y los mecanismos de protección, para posteriormente explicar cómo puede combinarse con la paginación en arquitecturas x86. De esta manera, una dirección lógica puede transformarse progresivamente en una dirección lineal y finalmente en una dirección física.

Objetivo general

Analizar el funcionamiento de los mecanismos de segmentación y paginación utilizados para la gestión de memoria, así como su combinación en sistemas basados en arquitecturas x86.

Objetivos específicos

- Comprender el concepto de segmentación de memoria.
- Identificar los componentes de una dirección lógica.
- Explicar la función de los descriptores de segmento.
- Comprender el funcionamiento de las tablas GDT y LDT.
- Diferenciar segmentación y paginación.
- Explicar el proceso de conversión de una dirección lógica en una dirección física.
- Comprender la paginación de dos niveles.
- Identificar algunos mecanismos de protección asociados con la segmentación.

Segmentación de memoria

La segmentación es un mecanismo de gestión de memoria mediante el cual el espacio lógico de un proceso se divide en diferentes segmentos de tamaño variable.

Un programa puede estar compuesto, por ejemplo, por:

- Segmento de código.
- Segmento de datos.
- Segmento de pila o stack.
- Otros segmentos asociados al programa o a sus bibliotecas.

Una característica importante es que estos segmentos no necesitan encontrarse de forma consecutiva en la memoria física. Cada segmento puede ubicarse en una zona diferente, lo que permite aprovechar los espacios disponibles de la memoria.

A diferencia de la paginación, donde las unidades de memoria tienen un tamaño fijo, los segmentos poseen un tamaño variable determinado por las necesidades de cada área del programa.

Dirección lógica

En un sistema segmentado, una dirección lógica está formada conceptualmente por un selector de segmento y un desplazamiento.

El selector permite identificar el segmento que se desea utilizar, mientras que el desplazamiento indica la posición dentro de dicho segmento.

De manera simplificada:

Dirección lógica = selector de segmento + desplazamiento

El desplazamiento debe encontrarse dentro de los límites establecidos para el segmento. Si se intenta acceder a una posición fuera del límite permitido, el procesador puede generar una excepción debido a un acceso inválido.

Descriptores de segmento

Para cada segmento existe un descriptor que contiene información necesaria para su administración.

Entre los datos principales de un descriptor se encuentran:

- Dirección base del segmento.
- Límite o tamaño permitido.

- Tipo de segmento.
- Permisos de acceso.
- Información relacionada con los niveles de privilegio.

La dirección base indica dónde comienza el segmento, mientras que el límite establece hasta dónde puede accederse dentro de él.

De manera simplificada, la dirección lineal se obtiene mediante:

Dirección lineal = dirección base + desplazamiento

Antes de realizar esta operación se debe comprobar que el desplazamiento sea válido de acuerdo con el límite del segmento.

Tablas GDT y LDT

En la arquitectura x86 existen estructuras utilizadas para almacenar los descriptores de los segmentos.

Las principales son:

GDT (Global Descriptor Table): contiene descriptores que pueden utilizarse de manera global.

LDT (Local Descriptor Table): permite almacenar descriptores asociados a un contexto o proceso particular.

El selector de segmento contiene información que permite determinar qué tabla debe utilizarse y qué descriptor debe seleccionarse.

El proceso puede representarse de la siguiente manera:

Selector → GDT/LDT → Descriptor → Base y límite

Los registros asociados a estas tablas permiten al procesador conocer dónde se encuentran las estructuras correspondientes en memoria.

Protección de memoria

La segmentación también permite establecer mecanismos de protección. Los descriptores pueden definir diferentes tipos de segmentos y permisos de acceso.

Por ejemplo, un segmento utilizado para almacenar código puede establecerse con permisos que impidan su modificación, mientras que un segmento de datos puede permitir operaciones de lectura y escritura.

Estos mecanismos ayudan a evitar que un proceso acceda o modifique áreas de memoria que no le corresponden.

También existen diferentes niveles de privilegio que permiten controlar qué operaciones pueden realizarse desde determinados niveles de ejecución.

Paginación

La paginación es otro mecanismo de gestión de memoria. A diferencia de la segmentación, utiliza unidades de tamaño fijo denominadas páginas para el espacio lógico y marcos para la memoria física.

Las páginas pueden ubicarse en diferentes marcos de memoria sin necesidad de que sean consecutivas.

Por ejemplo, las páginas de un proceso pueden encontrarse distribuidas de la siguiente manera:

Página 1 → Marco 7

Página 2 → Marco 2

Página 3 → Marco 10

Esto permite utilizar de manera más flexible la memoria disponible.

Segmentación y paginación combinadas

Una de las partes centrales de la clase es la combinación de ambos mecanismos.

En este modelo, primero se aplica la segmentación y posteriormente la paginación.

El proceso general es:

Dirección lógica → Segmentación → Dirección lineal → Paginación → Dirección física

Primero, el selector de segmento permite localizar el descriptor correspondiente. A partir de la información del descriptor se obtiene la dirección base y se suma el desplazamiento para obtener una dirección lineal.

Posteriormente, esa dirección lineal es procesada por el mecanismo de paginación. La dirección se divide en campos que permiten localizar la página correspondiente y el desplazamiento dentro de ella.

Finalmente, se obtiene el marco físico correspondiente y se combina con el desplazamiento para obtener la dirección física real en la memoria.

Paginación de dos niveles

En un sistema de 32 bits con páginas de 4 KiB, los 12 bits inferiores de una dirección pueden utilizarse como desplazamiento dentro de la página, debido a que:

$$4 \text{ KiB} = 2^{12} \text{ bytes}$$

Los bits restantes pueden utilizarse para identificar la página mediante estructuras de paginación.

En una paginación de dos niveles, de manera simplificada, la dirección puede dividirse de la siguiente forma:

10 bits → directorio de páginas

10 bits → tabla de páginas

12 bits → desplazamiento

El directorio permite localizar una tabla de páginas. Posteriormente, la tabla permite encontrar el marco físico correspondiente. Finalmente, el desplazamiento permite determinar la posición exacta dentro del marco.

Fragmentación

La utilización de estos mecanismos puede producir diferentes tipos de fragmentación.

En la segmentación, debido a que los segmentos tienen tamaños variables, puede producirse fragmentación externa, donde existen espacios libres entre diferentes bloques de memoria que pueden ser difíciles de aprovechar.

En la paginación puede producirse fragmentación interna, debido a que una página puede no utilizar completamente su espacio disponible.

Por ejemplo, si una página tiene un tamaño de 4 KiB y solamente se utilizan 3 KiB, queda aproximadamente 1 KiB sin utilizar dentro de esa página.

Importancia de la gestión de memoria

La segmentación y la paginación permiten que el sistema operativo administre la memoria de manera más flexible y segura.

La segmentación proporciona una visión lógica de la memoria, separando diferentes partes de un programa y permitiendo establecer límites y permisos. La paginación, por su parte, permite distribuir las diferentes partes del proceso en marcos físicos sin exigir que ocupen posiciones consecutivas.

La combinación de ambos mecanismos permite separar la organización lógica de un programa de su ubicación física en la memoria.

Conclusiones

La segmentación es una técnica de gestión de memoria que divide un proceso en segmentos de tamaño variable, como código, datos y pila. Cada segmento posee información que permite determinar su ubicación, tamaño y permisos de acceso.

La dirección lógica utilizada en un sistema segmentado contiene un selector de segmento y un desplazamiento. Mediante el selector se localiza el descriptor correspondiente en estructuras como la GDT o la LDT. A partir del descriptor se obtiene la dirección base y el límite del segmento, permitiendo calcular una dirección lineal y verificar que el acceso sea válido.

Mantenemos la estructura de un sistema que determina cuando un proceso se le asigna un espacio para poder operar de forma eficiente, lo que ha permitido la facilitación de la computación mucho antes que exista esta.

La paginación utiliza páginas y marcos de tamaño fijo, permitiendo ubicar las diferentes partes de un proceso en posiciones no consecutivas de la memoria física. Esto facilita la administración de la memoria y permite aprovechar los espacios disponibles.

Finalmente, la combinación de segmentación y paginación permite realizar una transformación en varias etapas: primero se convierte una dirección lógica en una dirección lineal mediante la segmentación y posteriormente esa dirección lineal se convierte en una dirección física mediante la paginación.

Por lo tanto, comprender el recorrido dirección lógica → segmentación → dirección lineal → paginación → dirección física es fundamental para entender cómo los sistemas operativos y las arquitecturas de procesadores administran y protegen la memoria.